TinyFaceMatch FPGA Implementation Report

A detailed implementation-level explanation of the PyTorch model, weight conversion, HLS accelerator, and PYNQ runtime

Project root:

`/mnt/OS/ramse/hls4ml_projects/tinyFaceNet`

Generated for the final DDR + integer-MAC HLS implementation

May 2026

Contents

1 Purpose of This Report **3**

2 End-to-End System View **3**
2.1 The Pipeline . . . 3
2.2 What "Embedding" Means . . . 4

3 Repository Components **4**
3.1 PyTorch Implementation Files . . . 4
3.2 HLS Implementation Files . . . 5
3.3 Processor-Side Runtime Files . . . 5

4 The Neural Network Architecture **5**
4.1 Why This Model Shape Was Chosen . . . 5
4.2 Important Building Blocks . . . 6
4.2.1 Standard Convolution . . . 6
4.2.2 Depthwise Convolution . . . 6
4.2.3 Pointwise 1 by 1 Convolution . . . 6
4.2.4 Depthwise-Separable Block . . . 6
4.2.5 Residual Bottleneck Block . . . 6
4.2.6 BatchNorm and PReLU . . . 7
4.3 Layer-by-Layer Shape Flow . . . 7

5 Training-Side Design **7**
5.1 Dataset Format . . . 7
5.2 Training Model Versus Inference Model . . . 8
5.3 Why ArcFace Is Used . . . 8

6 Weight Preparation **8**
6.1 Why Weights Need a Conversion Step . . . 8
6.2 BatchNorm Folding . . . 9
6.3 Quantization . . . 9
6.4 Packed Weight Layout . . . 9

7 What Scratch Memory Means **10**
7.1 The Simple Definition . . . 10
7.2 Why Scratch Memory Is Needed . . . 10
7.3 Why Not Put All Scratch Memory On-Chip? . . . 11
7.4 Why There Are Three Scratch Buffers . . . 11
7.5 Scratch Memory Versus Cache . . . 11

8 HLS Top-Level Interface **12**
8.1 What HLS Does . . . 12
8.2 Top Function Arguments . . . 12
8.3 AXI-Lite Control . . . 12
8.4 AXI Master Memory Ports . . . 13

9 Quantized Arithmetic in the Final HLS Version **13**
9.1 Why the Integer-MAC Version Exists . . . 13
9.2 Saturation . . . 13
9.3 Input Pixel Normalization in Hardware . . . 14

10 HLS Kernel Implementation **14**
10.1 `run_conv_image` . . . 14
10.2 `run_depthwise` . . . 15
10.3 `run_conv1x1` . . . 15
10.4 Residual Add Inside `run_conv1x1` . . . 15
10.5 `run_dense_l2` . . . 16

11 The Hard-Coded Layer Schedule **16**
11.1 Why It Is Hard-Coded . . . 16
11.2 Why This Beats a Generic Layer Runner . . . 17

12 Why the First Designs Failed and the Final One Worked **17**
12.1 The Original Resource Problem . . . 17
12.2 First Major Fix: DDR Scratch Buffers . . . 17
12.3 Second Fix: Specialized Hot Kernels . . . 17
12.4 Final Fix: Integer MAC Arithmetic . . . 18

13 Processor-Side Runtime **18**
13.1 What the ARM Processor Does . . . 18
13.2 Why Physical Addresses Matter . . . 19
13.3 Flush and Invalidate . . . 19
13.4 Starting the IP . . . 19

14 Camera and Verification Flow **19**
14.1 Live Frame Flow . . . 19
14.2 Detection Throttling . . . 20
14.3 No-Face Case . . . 20

15 Cosine Similarity Accelerator **20**
15.1 Software Cosine . . . 20
15.2 Hardware Cosine . . . 20

16 Measured Final Behavior **21**
16.1 Hardware Latency . . . 21
16.2 Accuracy Sanity Check . . . 21
16.3 Resource Direction . . . 21

17 Implementation Thought Process **21**
17.1 Step 1: Make the Model Exportable . . . 21
17.2 Step 2: Remove Training-Only Operations . . . 22
17.3 Step 3: Quantize in a Controlled Way . . . 22
17.4 Step 4: Use DDR for Large Tensors . . . 22
17.5 Step 5: Optimize the Real Hot Path . . . 22

18 Limitations and Future Improvement Paths **22**
18.1 Current Main Limitation . . . 22
18.2 Potential Improvements . . . 23

19 Final Mental Model **23**

1 Purpose of This Report

This report explains the implementation from the point of view of someone who wants to understand what was built, why it was built this way, and how the code maps to the hardware behavior. It is not mainly a command reference. The important question here is not "which command runs synthesis?" but rather:

- What is the neural network doing?
- What parts of the Python model survive into FPGA inference?
- What does the weight conversion script actually change?
- What is scratch memory, and why did the implementation need it?
- How does the HLS C++ code turn a CNN into hardware?
- Why did the final integer-MAC version become both faster and smaller?
- What does the processor-side Python code do around the accelerator?

The final system is a face verification pipeline. It takes a face crop, converts it into a 112 by 112 RGB image, runs a compact CNN on the FPGA fabric, produces a 128-dimensional embedding vector, and compares that embedding against enrolled embeddings using cosine similarity.

2 End-to-End System View

2.1 The Pipeline

The implementation is split into four conceptual layers:

Layer	Responsibility	Main files
Training side	Learn face-discriminative features from image folders.	`tinyfacematch_pytorch/model.py`, `data.py`, `train.py`
Conversion side	Fold and quantize learned weights into a binary layout understood by HLS.	`tinyfacematch_pytorch/prepare_hls_weights.py`
FPGA side	Execute the CNN forward pass using HLS-generated hardware.	`hls_ddr_intmac/tinyfacematch_ip.cpp`, `tinyfacematch_ip.hpp`, `weights_offsets.hpp`
Processor side	Load bitstream, allocate memory, feed images, read embeddings, run camera/matching logic.	`scripts/pynq_hls_camera_verify.py`, notebook cells

The most important design boundary is this:

> The FPGA CNN IP does not know about cameras, identities, enrollment, files, or authorization. It only knows how to transform:
>
> $$\text{112x112 RGB image} + \text{packed weights} + \text{scratch buffers} \rightarrow \text{128-D embedding}$$
>
> Everything else is handled by Python on the ARM processor.

2.2 What "Embedding" Means

An embedding is a vector representation of a face. Instead of saying "this image is class 17," the model outputs 128 numbers. Those 128 numbers are trained so that images of the same person produce nearby vectors, while images of different people produce vectors pointing in different directions.

The final embedding is L2-normalized. That means its Euclidean length is 1:

$$\|e\|_2 = 1$$

Once vectors are normalized, the dot product between two embeddings is also their cosine similarity:

$$\cos(a, b) = a \cdot b$$

That is why verification can be implemented as:

```
score = dot(camera_embedding, authorized_embedding)
if score >= threshold:
    MATCH
else:
    NOT MATCH
```

The CNN does not directly output "match" or "not match." It outputs an embedding. Matching is a separate step.

3 Repository Components

3.1 PyTorch Implementation Files

`tinyfacematch_pytorch/model.py`
: Defines the neural network architecture. The core inference model is `TinyFaceMatch128`. The training wrapper is `TinyFaceMatchTrainingModel`, which adds an ArcFace classifier during training only.

`tinyfacematch_pytorch/data.py`
: Defines the folder-based dataset loader. It reads identity folders, loads RGB images, resizes them to 112 by 112, and normalizes pixels with:

$$x = \frac{\text{pixel} - 127.5}{128.0}$$

`tinyfacematch_pytorch/train.py`
: Implements training. It creates train/validation splits, runs the optimizer, uses ArcFace-style logits, saves checkpoints, and records metrics.

`tinyfacematch_pytorch/prepare_hls_weights.py`
: Converts a PyTorch checkpoint into the exact packed integer format used by the HLS design. This is the bridge between machine learning code and FPGA inference code.

`tinyfacematch_pytorch/eval_hls_prepared.py`
: Replays the quantized/folded network in PyTorch. This makes it possible to check whether the prepared HLS weights still behave like the original trained model before putting the design on hardware.

3.2 HLS Implementation Files

`hls_ddr_intmac/tinyfacematch_ip.hpp`
Declares constants, fixed-point types, and the top-level HLS function. This file tells Vitis HLS the C++ signature of the accelerator.

`hls_ddr_intmac/tinyfacematch_ip.cpp`
Contains the actual CNN implementation. It defines the top AXI interfaces, helper functions for quantized arithmetic, convolution kernels, depthwise kernels, dense output logic, and the hard-coded layer execution order.

`hls_ddr_intmac/weights_offsets.hpp`
Generated header containing byte offsets for each layer's parameters inside `weights.bin`. The HLS code uses these offsets to find each layer's weights, biases, and PReLU parameters.

`hls_ddr_intmac/testbench.cpp`
C simulation testbench. It loads a known input, runs the HLS top function, and compares the output against an expected vector.

3.3 Processor-Side Runtime Files

`scripts/pynq_hls_camera_verify.py`
Shows the complete runtime pattern on PYNQ: load overlay, allocate buffers, write physical addresses into HLS registers, prepare face crops, start the IP, wait for completion, read the output embedding, and compare against an enrolled profile.

Notebook cells
During testing, notebook cells used the same ideas as the script but were adjusted for live DroidCam input, dummy latency tests, and the separate cosine-similarity IP.

4 The Neural Network Architecture

4.1 Why This Model Shape Was Chosen

The model is not a large face recognition network. It is a compact embedding network designed to fit on a small FPGA, especially a Zynq-7020 class device. That means the architecture has to balance:

- accuracy good enough for face verification,
- a small number of channels,
- mostly convolution operations that can be expressed predictably in HLS,
- low enough LUT/FF/BRAM usage to place in Vivado,
- a fixed input size so the HLS implementation can be static.

The inference model is `TinyFaceMatch128`. It accepts one RGB face image with shape:

$$112 \times 112 \times 3$$

and produces:

$$128$$

embedding values.

4.2 Important Building Blocks

4.2.1 Standard Convolution

A convolution layer slides a small filter over the image or feature map. For every output pixel, it multiplies nearby input values by learned weights and sums them. In simplified form:

$$y[o, y, x] = b[o] + \sum_{i,ky,kx} x[i, y + ky, x + kx] \cdot w[o, i, ky, kx]$$

The first layer, `conv0`, is a normal 3 by 3 convolution. It converts raw RGB pixels into 16 feature channels and also downsamples the image from 112 by 112 to 56 by 56.

4.2.2 Depthwise Convolution

A depthwise convolution is a cheaper form of convolution. Instead of mixing all input channels into all output channels, each channel gets its own spatial filter.

For example, if there are 64 channels, a 3 by 3 depthwise convolution uses 64 separate 3 by 3 filters. Channel 0 only filters channel 0, channel 1 only filters channel 1, and so on. It captures local spatial patterns but does not mix information between channels.

This is cheaper than a full convolution because the number of multiplies is much smaller. The tradeoff is that another layer is needed to mix channels.

4.2.3 Pointwise 1 by 1 Convolution

A pointwise convolution is a 1 by 1 convolution. It looks at one spatial location at a time and mixes the channels at that location.

If the input at one pixel is a vector of 64 channels, a 1 by 1 convolution computes new channel vectors by matrix-vector multiplication. This is how the model changes channel count and combines information after depthwise filtering.

4.2.4 Depthwise-Separable Block

A depthwise-separable block combines:

1. a depthwise 3 by 3 convolution for spatial filtering,
2. a pointwise 1 by 1 convolution for channel mixing.

This is much cheaper than a full 3 by 3 convolution that mixes all input channels into all output channels at every spatial position.

4.2.5 Residual Bottleneck Block

A residual bottleneck block has:

1. **expand 1 by 1 convolution:** increase channel count,
2. **depthwise 3 by 3 convolution:** spatial filtering,
3. **project 1 by 1 convolution:** reduce channel count back down,
4. **optional residual add:** add the original input back to the block output.

The residual connection is only used when input and output have the same shape. For example, a block that maps 28 by 28 by 32 to 28 by 28 by 32 can add its input to its output. A block that changes resolution or channel count cannot do that directly.

Residuals help training because the block can learn a correction to the input rather than learning the whole transformation from scratch.

4.2.6 BatchNorm and PReLU

During training, many convolution layers are followed by BatchNorm and PReLU.

BatchNorm stores learned scale and shift parameters, plus running mean and variance. At inference time, BatchNorm is just a linear transformation:

$$y = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Because it is linear, it can be folded into the previous convolution weights and biases. That means the FPGA does not need a separate BatchNorm hardware stage.

PReLU is an activation function:

$$y = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases}$$

Unlike ReLU, which clips negative values to zero, PReLU keeps a scaled negative value. The learned α values are exported as part of the packed weights.

4.3 Layer-by-Layer Shape Flow

The final implementation hard-codes this network flow. The table below uses height by width by channels.

Stage	Operation	Output shape	Implementation idea
Input	RGB face crop	112 x 112 x 3	uint8 image in DDR
conv0	3x3 stride-2 conv + PReLU	56 x 56 x 16	`run_conv_image`
dwsep0_dw	3x3 depthwise	56 x 56 x 16	`run_depthwise`
dwsep0_pw	1x1 pointwise	56 x 56 x 32	`run_conv1x1`
b0 expand	1x1 conv	56 x 56 x 64	`run_conv1x1`
b0 depthwise	3x3 stride-2 depthwise	28 x 28 x 64	`run_depthwise`
b0 project	1x1 conv	28 x 28 x 32	no residual
b1	bottleneck	28 x 28 x 32	residual add
b2	bottleneck	28 x 28 x 32	residual add
b3	stride-2 bottleneck	14 x 14 x 64	no residual
b4	bottleneck	14 x 14 x 64	residual add
b5	bottleneck	14 x 14 x 64	residual add
b6	stride-2 bottleneck	7 x 7 x 64	no residual
b7	bottleneck	7 x 7 x 64	residual add
conv_final	1x1 conv	7 x 7 x 128	channel expansion
global_dw7	7x7 depthwise	1 x 1 x 128	spatial collapse
dense	linear + L2 normalize	128	final embedding

5 Training-Side Design

5.1 Dataset Format

The training dataset is folder-per-identity. Conceptually:

```
data_root/
  person_000/
    image1.jpg
```

```
    image2.jpg
  person_001/
    image1.jpg
    image2.jpg
```

`FaceFolderDataset` assigns each identity folder a numeric label. Every image is:

1. opened as RGB,
2. optionally augmented,
3. resized to 112 by 112,
4. converted to float32,
5. normalized with $(pixel - 127.5)/128.0$,
6. returned in CHW layout for PyTorch.

CHW means channels-height-width. PyTorch convolution layers usually expect:

$$N \times C \times H \times W$$

where N is batch size.

5.2 Training Model Versus Inference Model

During training, the model is:

$$\text{backbone} + \text{ArcFace classifier}$$

The backbone is the CNN that produces embeddings. The ArcFace classifier is only a training tool. It forces embeddings of the same identity to cluster together and embeddings of different identities to separate angularly.

Only the backbone is exported to FPGA. The classifier is not needed at runtime because runtime verification compares embeddings with cosine similarity.

5.3 Why ArcFace Is Used

Normal classification training asks the model to predict an identity class. That is useful, but face verification needs a geometry where distances between embeddings are meaningful. ArcFace encourages this by applying an angular margin.

In plain terms:

- same-person embeddings are pushed closer together,
- different-person embeddings are pushed farther apart,
- the final embedding space becomes useful for cosine similarity.

The FPGA does not implement ArcFace. ArcFace only shapes the embedding space during training.

6 Weight Preparation

6.1 Why Weights Need a Conversion Step

The PyTorch checkpoint cannot be consumed directly by the HLS IP. The checkpoint contains PyTorch tensors, BatchNorm modules, floating-point values, and module names. The FPGA design expects one flat array of integer words called `weights.bin`.

The conversion step transforms:

`PyTorch checkpoint` → `folded fp32 arrays` → `quantized integer arrays` → `packed weights.bin + offsets header`

6.2 BatchNorm Folding

BatchNorm folding is one of the most important transformations. During inference, a convolution followed by BatchNorm can be replaced by a single convolution with adjusted weights and bias.

For a convolution output:

$$z = Wx + b$$

BatchNorm applies:

$$y = \gamma \frac{z - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

This can be rewritten as:

$$y = W'x + b'$$

where:

$$W' = W \cdot \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}}$$

and:

$$b' = \beta + (b - \mu)\frac{\gamma}{\sqrt{\sigma^2 + \epsilon}}$$

After this transformation, there is no separate BatchNorm layer in HLS. That saves hardware and simplifies the runtime flow.

6.3 Quantization

Quantization means storing real numbers as scaled integers. For example, a value $x = 0.25$ with 12 fractional bits becomes:

$$\text{integer} = round(0.25 \cdot 2^{12}) = 1024$$

To recover the approximate real value:

$$x \approx \frac{1024}{2^{12}} = 0.25$$

The final implementation uses:

Quantity	Stored type	Fraction bits	Meaning
Weights	int16	12	learned convolution/dense weights
Biases	int32	20	folded convolution/dense biases
PReLU alpha	int16	12	learned negative-slope values
Intermediate activations	int16	10	feature maps between layers
Output embedding	int16	14	normalized 128-D output

The notation Q6.10 is useful here. It means a fixed-point number with 16 total bits, 6 integer/sign bits, and 10 fractional bits. The HLS activation type `ap_fixed<16,6>` corresponds to that idea.

6.4 Packed Weight Layout

The final `weights.bin` file is a flat sequence of 16-bit words. For each layer, the data is packed in this order:

1. quantized weights,
2. quantized biases,
3. optional PReLU alpha values.

Biases are int32, but the memory bus and weight stream are int16 words. So each int32 bias is stored as two int16 words. The HLS helper `read_i32_words` reconstructs the int32 value from the two 16-bit pieces.

`weights_offsets.hpp` tells HLS where each layer begins inside the packed array. For example:

```
static constexpr int TFM_CONV0_OFFSET_BYTES = 0;
static constexpr int TFM_DWSEP0_DW_OFFSET_BYTES = 960;
...
static constexpr int TFM_DENSE_OFFSET_BYTES = 225408;
```

The top-level HLS code divides byte offsets by 2 because the `weights` pointer is an `ap_int<16>*`. One address step is one 16-bit word, not one byte.

7 What Scratch Memory Means

7.1 The Simple Definition

Scratch memory is temporary workspace. It is not model weights. It is not final output. It is the memory used to hold intermediate feature maps while the CNN is running.

A CNN is a sequence of layers. Each layer consumes one tensor and produces another tensor. For example:

$$\text{conv0 output} = 56 \times 56 \times 16$$

That output must be stored somewhere because the next layer needs to read it. That "somewhere" is scratch memory.

7.2 Why Scratch Memory Is Needed

The model is not just one operation. It is a chain:

$$\texttt{image} \rightarrow \texttt{conv0 output} \rightarrow \texttt{dwsep0 output} \rightarrow \texttt{bottleneck outputs} \rightarrow \texttt{embedding}$$

Every arrow produces intermediate data. These intermediate tensors are too large to keep entirely in local FPGA registers. Some examples:

Tensor shape	int16 words	Approx bytes
56 x 56 x 16	50,176	100 KB
56 x 56 x 32	100,352	201 KB
56 x 56 x 64	200,704	401 KB
28 x 28 x 64	50,176	100 KB
14 x 14 x 128	25,088	50 KB

The largest intermediate tensor in the implemented network is:

$$64 \times 56 \times 56 = 200704 \text{ int16 words}$$

Since each int16 is 2 bytes:

$$200704 \times 2 = 401408 \text{ bytes}$$

So one large scratch buffer is about 401 KB.

7.3 Why Not Put All Scratch Memory On-Chip?

The Zynq-7020 FPGA fabric has limited BRAM. BRAM is fast on-chip block RAM, but there is not enough of it to store all intermediate CNN tensors naively.

The early HLS designs tried to keep too much tensor storage inside the fabric. That caused over-utilization in Vivado. The original resource reports showed the design needing far more LUTs, flip-flops, muxes, and BRAM than the device had. That is why place design failed.

The solution was to move large intermediate tensors into external DDR memory. DDR is slower than BRAM, but much larger. On a PYNQ/Zynq board, DDR is shared between the ARM processor and programmable logic. The HLS IP accesses DDR through AXI master ports.

7.4 Why There Are Three Scratch Buffers

The final top function receives:

```
ap_int<16>* scratch0
ap_int<16>* scratch1
ap_int<16>* scratch2
```

These are three temporary tensor buffers in DDR. They are reused throughout the network.

The implementation does not allocate a separate buffer for every layer. That would waste memory. Instead, it ping-pongs between scratch buffers.

For example:

```
run_conv_image(image_rgb, scratch0, ...)
run_depthwise(scratch0, scratch1, ...)
run_conv1x1(scratch1, scratch0, ...)
```

This means:

- layer 1 writes to `scratch0`,
- layer 2 reads `scratch0` and writes `scratch1`,
- layer 3 reads `scratch1` and writes back to `scratch0`.

`scratch2` is useful for bottleneck blocks because some blocks need an input tensor preserved for residual addition while another intermediate tensor is being computed.

7.5 Scratch Memory Versus Cache

Scratch memory and cache are not the same thing.

Scratch memory
: Explicit temporary memory that the program controls. In this project, the Python runtime allocates scratch buffers, passes their physical addresses to the IP, and the HLS code reads/writes them directly.

Cache
: Hardware-managed memory used automatically to speed up repeated reads. The programmer usually does not address cache directly.

In this project, scratch buffers are intentional, visible parts of the hardware/software contract.

8 HLS Top-Level Interface

8.1 What HLS Does

HLS means High-Level Synthesis. Instead of writing Verilog manually, the design is written in C++ with HLS-specific types and pragmas. Vitis HLS converts that C++ into RTL hardware.

The top function:

```
void tinyfacematch_ip(
    const ap_uint<8>* image_rgb,
    const ap_int<16>* weights,
    ap_int<16>* scratch0,
    ap_int<16>* scratch1,
    ap_int<16>* scratch2,
    ap_int<16>* embedding_out
);
```

becomes an IP block in Vivado.

8.2 Top Function Arguments

Argument	Direction	Meaning
`image_rgb`	input	flattened 112 x 112 x 3 uint8 RGB image
`weights`	input	packed quantized model weights
`scratch0`	temporary read/write	DDR workspace for feature maps
`scratch1`	temporary read/write	DDR workspace for feature maps
`scratch2`	temporary read/write	DDR workspace for feature maps/residuals
`embedding_out`	output	128 int16 output embedding values

8.3 AXI-Lite Control

AXI-Lite is a simple memory-mapped register interface. The processor uses it to configure and control the IP.

In this design, AXI-Lite is used for:

- writing the physical address of `image_rgb`,
- writing the physical address of `weights`,
- writing the physical addresses of `scratch0`, `scratch1`, and `scratch2`,
- writing the physical address of `embedding_out`,
- starting the IP,
- polling until the IP is done.

The Python code writes addresses into offsets such as:

```
IMAGE_OFFSET = 0x10
WEIGHTS_OFFSET = 0x1C
SCRATCH0_OFFSET = 0x28
SCRATCH1_OFFSET = 0x34
SCRATCH2_OFFSET = 0x40
OUT_OFFSET = 0x4C
```

The control register at offset `0x00` contains bits such as `ap_start` and `ap_done`.

8.4 AXI Master Memory Ports

AXI master ports let the IP initiate reads and writes to DDR. That is how the accelerator reads images and weights and writes scratch tensors.

The HLS pragmas look like:

```
#pragma HLS INTERFACE m_axi port=image_rgb bundle=gmem0
#pragma HLS INTERFACE m_axi port=weights bundle=gmem1
#pragma HLS INTERFACE m_axi port=scratch0 bundle=gmem2
#pragma HLS INTERFACE m_axi port=scratch1 bundle=gmem2
#pragma HLS INTERFACE m_axi port=scratch2 bundle=gmem2
#pragma HLS INTERFACE m_axi port=embedding_out bundle=gmem2
```

The important idea:

- AXI-Lite passes addresses and control bits.
- AXI master ports move the actual bulk data.

9 Quantized Arithmetic in the Final HLS Version

9.1 Why the Integer-MAC Version Exists

Earlier HLS versions used fixed-point values more directly inside convolution loops. That was correct, but the generated hardware was heavier than necessary.

The final version recognizes that convolution inputs and weights are already stored as integers:

- activations are stored as int16 with 10 fractional bits,
- weights are stored as int16 with 12 fractional bits,
- biases are stored as int32 with 20 fractional bits.

Therefore, a convolution multiply can be done as an integer multiply:

$$\text{act_q10} \times \text{weight_q12} = \text{acc_q22}$$

The result has:

$$10 + 12 = 22$$

fractional bits.

Bias has 20 fractional bits, so the code shifts the bias left by 2 bits to align it with Q22:

$$\text{bias_q22} = \text{bias_q20} << 2$$

After accumulation, the result is shifted down by 12 bits to return to activation Q10:

$$\text{out_q10} = \mathit{round}(\text{acc_q22}/2^{12})$$

This is exactly why the helper is named:

```
round_shift_sat_i16(acc, TFM_WEIGHT_FRAC_BITS)
```

Since `TFM_WEIGHT_FRAC_BITS = 12`, it removes the weight fractional scale after multiplication.

9.2 Saturation

Saturation means clamping a value to the valid int16 range. The valid range is:

$$-32768 \leq x \leq 32767$$

If an accumulated result is too large, it becomes 32767. If it is too negative, it becomes -32768.

This avoids wraparound. Wraparound would be disastrous because a large positive value could become a negative value just because of integer overflow.

9.3 Input Pixel Normalization in Hardware

The training input normalization is:

$$x = \frac{pixel - 127.5}{128.0}$$

The HLS activation format uses 10 fractional bits. So the hardware wants:

$$x_q10 = x \cdot 2^{10}$$

Substitute:

$$x_q10 = \frac{pixel - 127.5}{128.0} \cdot 1024$$

Since $1024/128 = 8$:

$$x_q10 = pixel \cdot 8 - 127.5 \cdot 8$$

$$x_q10 = pixel \cdot 8 - 1020$$

That is why the HLS code uses:

```
return (ap_int<16>)(pixel * 8 - 1020);
```

This is a nice example of replacing floating-point preprocessing with exact integer arithmetic.

10 HLS Kernel Implementation

10.1 `run_conv_image`

`run_conv_image` handles the first convolution. It is special because its input is not an int16 feature map yet. Its input is raw uint8 RGB image data.

Responsibilities:

- read RGB pixels,
- convert them into Q10 normalized activations,
- apply 3 by 3 convolution,
- apply folded bias,
- shift result back to Q10,
- apply PReLU,
- write a 56 x 56 x 16 output tensor to scratch memory.

The kernel caches one output channel's 3 by 3 by 3 weights locally:

```
ap_int<16> w_cache[3 * 3 * 3];
```

This is small. Caching it avoids repeatedly fetching the same weights from DDR for every output pixel.

10.2 `run_depthwise`

`run_depthwise` handles both ordinary 3 by 3 depthwise layers and the final 7 by 7 global depthwise layer.

For each channel, it:

1. loads that channel's kernel weights into a small local cache,
2. reads the matching input channel from scratch memory,
3. applies the spatial convolution,
4. adds bias,
5. optionally applies PReLU,
6. writes the output channel.

Depthwise convolution is efficient because it does not loop over every input channel for every output channel. It only processes one input channel for one output channel.

10.3 `run_conv1x1`

`run_conv1x1` implements pointwise convolutions. These are important because most channel mixing in the network happens here.

At each spatial location, a 1 by 1 convolution is essentially:

$$\text{output vector} = W \cdot \text{input vector} + b$$

The implementation processes two output channels at a time:

```
for (int oc = 0; oc < out_c; oc += 2) {
    ap_int<16> w_cache0[TFM_MAX_CHANNELS];
    ap_int<16> w_cache1[TFM_MAX_CHANNELS];
    ...
}
```

This gives more parallel work per input read. For one input channel value, the code updates two accumulators:

```
acc0 += in_value * w_cache0[ic];
acc1 += in_value * w_cache1[ic];
```

This costs more multipliers than a one-output-channel loop, but it reduces repeated input memory reads and improves latency. The final design still fit the FPGA after this because the integer-MAC rewrite reduced LUT pressure.

10.4 Residual Add Inside `run_conv1x1`

Some project layers are the end of residual bottleneck blocks. For those layers, the output of the project convolution is added to the original block input.

The HLS function supports this with:

```
bool add_residual
```

When enabled:

```
out0 = add_q10_sat(out0, read_act_raw(residual, oc, oy, ox, ...));
```

Both tensors are already Q10 activations, so the add does not need rescaling. It only needs saturation.

10.5 `run_dense_l2`

`run_dense_l2` is the final stage. It reads the 1 by 1 by 128 tensor after the global depthwise layer, applies the 128 by 128 dense layer, and normalizes the result.

This stage still uses fixed-point helper types rather than pure integer MAC everywhere. That is acceptable because the dense stage is not the main runtime bottleneck compared with the convolutional body.

The final output is quantized with 14 fractional bits:

$$\text{embedding_int16} = \mathit{round}(\text{embedding_float} \cdot 2^{14})$$

The processor-side Python code divides by 2^{14} and normalizes again for safety.

11 The Hard-Coded Layer Schedule

11.1 Why It Is Hard-Coded

The HLS implementation is not a general neural network interpreter. It does not parse ONNX. It does not read a dynamic graph. It is a static accelerator for this one network architecture.

That is intentional. A static schedule lets HLS optimize loops and memory access patterns. It also avoids building hardware for features the design does not need.

The top function calls kernels in the exact network order:

```
run_conv_image(...);
run_depthwise(...);
run_conv1x1(...);

run_conv1x1(...); // b0 expand
run_depthwise(...); // b0 depthwise
run_conv1x1(...); // b0 project

...

run_conv1x1(...); // conv_final
run_depthwise(...); // global_dw7
run_dense_l2(...);
```

Each call passes:

- input scratch pointer,
- output scratch pointer,
- residual pointer if needed,
- channel count,
- spatial size,
- stride/padding/kernel parameters,
- packed-weight offset,
- whether PReLU is present,
- whether residual addition is present.

11.2 Why This Beats a Generic Layer Runner

A generic implementation would have to store layer descriptors, switch between operation types, handle arbitrary shapes, and make conservative assumptions. That flexibility costs hardware.

This implementation chooses the opposite tradeoff:

- less flexible,
- much simpler hardware,
- predictable resource use,
- easier to debug against the PyTorch architecture,
- good fit for one deployed model.

12 Why the First Designs Failed and the Final One Worked

12.1 The Original Resource Problem

The early HLS implementation was correct enough for simulation, but too large for the target device. Vivado reported over-utilization in LUTs, flip-flops, muxes, carry chains, and BRAM.

That means the design did not fail because of a syntax problem. It failed because the generated hardware physically could not fit.

The core issue was that HLS was mapping too much tensor storage and fixed-point computation into fabric resources. CNN intermediate tensors are large, and a naive hardware mapping can explode quickly.

12.2 First Major Fix: DDR Scratch Buffers

The first viability fix was to move intermediate tensors to DDR scratch buffers.

Before:

$$\text{large intermediate tensors} \rightarrow \text{mostly on-chip storage pressure}$$

After:

$$\text{large intermediate tensors} \rightarrow \text{external DDR scratch buffers}$$

This reduced BRAM/LUT pressure enough for the design to become placeable. The cost was latency because DDR access is slower than local storage.

12.3 Second Fix: Specialized Hot Kernels

The next version specialized common operations:

- a dedicated image convolution for the first layer,
- a dedicated 1 by 1 convolution kernel,
- local weight caching,
- two output channels at a time for pointwise convolution.

This reduced repeated memory work and made the design more direct.

12.4 Final Fix: Integer MAC Arithmetic

The final `hls_ddr_intmac` version changed the arithmetic model in the convolution-heavy parts.

Instead of repeatedly converting int16 values into fixed-point objects in the inner loops, it keeps the hot path as integer multiply-accumulate:

```
acc += (ap_int<48>)in_value * (ap_int<48>)weight;
```

Then it performs one rounded shift and saturation at the end.

This improved both speed and resource use because:

- integer multiplies map cleanly to DSPs,
- less fixed-point conversion logic is generated,
- fewer LUTs are needed around arithmetic,
- the math still matches the quantized model closely.

The slightly surprising result was that the design became faster while also using fewer LUTs. That is not dubious. It happens when a rewrite gives HLS a simpler hardware structure to synthesize.

13 Processor-Side Runtime

13.1 What the ARM Processor Does

On a PYNQ/Zynq board, the system has:

- PS: Processing System, the ARM processor running Linux/Python,
- PL: Programmable Logic, the FPGA fabric containing the HLS IP.

The processor handles all high-level application logic:

- load the bitstream,
- allocate physically contiguous buffers,
- load `weights.bin`,
- configure the HLS IP registers,
- capture camera frames,
- detect/crop faces,
- resize and format the image,
- start the accelerator,
- read the embedding,
- compare against authorized embeddings.

The FPGA handles the CNN math.

13.2 Why Physical Addresses Matter

Python objects normally have virtual addresses. The FPGA cannot use normal Python object addresses. The FPGA needs physical DDR addresses.

That is why PYNQ uses `allocate`. It creates buffers that:

- Python can read and write,
- the FPGA can access through physical addresses,
- can be flushed and invalidated for cache coherency.

The runtime writes physical addresses into the HLS IP:

```
write_u64(ip, IMAGE_OFFSET, image_buf.physical_address)
write_u64(ip, WEIGHTS_OFFSET, weights_buf.physical_address)
write_u64(ip, SCRATCH0_OFFSET, scratch0.physical_address)
...
```

13.3 Flush and Invalidate

Flush and invalidate are cache-coherency operations.

Flush
: Push CPU-written data out to DDR so the FPGA sees the latest values. The image buffer and weights buffer must be flushed before the IP reads them.

Invalidate
: Discard stale CPU cache lines so Python reads fresh data from DDR after the FPGA writes it. The output buffer must be invalidated after the IP finishes.

Without these operations, the accelerator might read old data or Python might read old output.

13.4 Starting the IP

The HLS control register is written with `0x01` to set `ap_start`. Then Python polls until `ap_done` is set:

```
ip.write(CTRL_OFFSET, 0x01)
while (ip.read(CTRL_OFFSET) & 0x2) == 0:
    pass
```

This is a blocking call. The Python code waits there until the CNN finishes.

14 Camera and Verification Flow

14.1 Live Frame Flow

The live notebook/script flow is:

1. Open DroidCam/OpenCV video stream.
2. Grab frames in a thread or loop.
3. Throttle detection so face detection does not run every single frame.
4. Detect the largest face with Haar cascade.
5. Crop the face region.

6. Convert BGR to RGB because OpenCV uses BGR but the model expects RGB.
7. Resize to 112 by 112.
8. Flatten into HWC RGB uint8 layout.
9. Run the HLS CNN IP.
10. Convert int16 embedding output back to float.
11. Compare with authorized embeddings.

14.2 Detection Throttling

Face detection is not run on every loop iteration. It is throttled using a frame counter, for example:

```
if frame_count % DETECTION_EVERY == 0:
    last_detections = detect_and_embed_hls(frame)
```

The reason is simple: Haar detection and CNN inference are expensive compared with just reading a frame. Throttling lets the live loop stay responsive. The last known detection can be reused between detection intervals.

14.3 No-Face Case

When no face is detected, the application can still run a dummy cosine comparison. That is not for recognition accuracy. It is useful when testing that the cosine IP path is alive and returns a low score.

The dummy vectors can be orthogonal:

$$a = [1, 0, 0, \ldots], \quad b = [0, 1, 0, \ldots]$$

Their dot product is zero, so the match result should be low.

15 Cosine Similarity Accelerator

15.1 Software Cosine

The simplest cosine check is done in NumPy:

```
score = float(np.dot(cam_emb, auth_emb))
match = score >= MATCH_THRESHOLD
```

Because embeddings are normalized, dot product equals cosine similarity.

15.2 Hardware Cosine

The separate cosine similarity IP replaces the software dot product. The CNN still produces the embedding. The cosine IP compares that embedding against authorized embeddings.

Conceptually:

$$\texttt{camera embedding} + \texttt{authorized embedding} \rightarrow \texttt{cosine score} + \texttt{match bit}$$

This is a clean split:

- TinyFaceMatch IP computes embeddings.
- Cosine IP computes match scores.
- Python controls the loop and threshold policy.

16 Measured Final Behavior

16.1 Hardware Latency

The final board measurement reported:

Measurement	Runs	Min	Mean	P95	Max
CNN only	20	1287.00 ms	1287.28 ms	1287.45 ms	1290.65 ms
Full embedding	20	1288.31 ms	1288.66 ms	1288.85 ms	1292.07 ms

The difference between CNN-only and full embedding is about:

$$1288.66 - 1287.28 = 1.38 \text{ ms}$$

That means almost all the runtime is inside the HLS CNN IP. The Python buffer setup, output conversion, and final normalization are tiny compared with the CNN itself.

16.2 Accuracy Sanity Check

The integer-MAC version passed simulation with:

- max int16 output error around 16,
- mean int16 output error around 4.7,
- cosine to expected around 0.999991.

That means the quantized HLS math tracks the expected embedding direction extremely closely. For face verification, preserving the direction of the embedding is more important than matching every raw component exactly.

16.3 Resource Direction

The final int-MAC version used roughly:

- BRAM_18K: 10,
- DSP: 50,
- FF: about 16.7k,
- LUT: about 27k.

This is the key achievement: the implementation became viable for the FPGA while keeping accuracy.

17 Implementation Thought Process

17.1 Step 1: Make the Model Exportable

The model had to be written in a way that could be represented statically. That means:

- fixed input size,
- fixed channel sizes,
- common convolution blocks,
- no dynamic Python-side control inside inference,
- output dimension fixed at 128.

This made it possible to manually map the network into HLS.

17.2 Step 2: Remove Training-Only Operations

The ArcFace classifier is useful for training but irrelevant for FPGA inference. BatchNorm is useful in training but can be folded away for inference.

So the deployed model is not the training model exactly. It is the inference backbone after BatchNorm folding and quantization.

17.3 Step 3: Quantize in a Controlled Way

Quantization was not left to the HLS compiler. It was done explicitly in Python so the exact numeric format was known:

- weights Q12,
- biases Q20,
- activations Q10,
- output Q14.

This made it possible to reason about the arithmetic and build the integer-MAC design.

17.4 Step 4: Use DDR for Large Tensors

The failed resource-heavy version showed that large intermediate tensors could not live comfortably on-chip. The design was changed to make DDR scratch buffers part of the interface.

This was the biggest architectural pivot. It traded memory bandwidth and latency for a design that actually fits.

17.5 Step 5: Optimize the Real Hot Path

The important runtime cost is convolution. So the final optimization focused on:

- avoiding unnecessary fixed-point object math in convolution loops,
- caching small weight slices locally,
- specializing 1 by 1 convolution,
- processing two pointwise output channels at a time.

Dense output and Python overhead were not the main bottlenecks, so they were not over-optimized first.

18 Limitations and Future Improvement Paths

18.1 Current Main Limitation

The current accelerator is viable and accurate, but the CNN still takes about 1.29 seconds per embedding on the measured board setup. That means the live system should throttle inference and cannot treat the CNN as a high-frame-rate accelerator yet.

18.2 Potential Improvements

More parallel output channels
`run_conv1x1` currently processes two output channels at a time. More parallelism could improve latency but would consume more DSPs and memory bandwidth.

Better local tiling
Instead of reading/writing large DDR feature maps in straightforward order, selected tiles could be kept on-chip. This is harder to implement but could reduce DDR traffic.

More aggressive layer fusion
Some producer/consumer layer pairs might be fused so intermediate results do not always round-trip to DDR. This is complex because the model has residuals and changing spatial sizes.

Smaller model variant
Reducing channels or blocks would directly reduce CNN latency. The cost would be accuracy and embedding quality.

Multiple accelerators or larger FPGA
A larger device would allow more parallelism and more on-chip buffering.

19 Final Mental Model

The implementation can be understood as a carefully constrained translation:

> PyTorch learns a compact face embedding model. The conversion script folds BatchNorm, quantizes the learned parameters, and packs them into a flat memory image. The HLS IP is a static hardware implementation of that exact model. It uses DDR scratch buffers to store intermediate tensors because the FPGA fabric cannot hold all feature maps on-chip. The final integer-MAC kernels exploit the known quantization format to make convolution hardware simpler, faster, and smaller. The PYNQ Python side owns the application: camera input, face detection, buffer management, accelerator control, and match decisions.